

Este proyecto está diseñado para simular con precisión un entorno laboral real , donde el objetivo central es la refactorización, modernización, automatización y despliegue continuo de un sistema legado basado en Python/Flask.

El desarrollo se divide en 6 fases incrementales , cada una con entregables específicos que el alumno debe consignar en su repositorio institucional de GitHub.

Fase 1: Ingeniería Reversa y Refactorización (Código Limpio)

Analizar el código original monolítico y reestructurarlo bajo una arquitectura modular y escalable.

Entregables Obligatorios:

Repositorio Modular en GitHub: Código fuente reestructurado aplicando el patrones de diseño, uso de Blueprints, segregación de modelos de datos y centralización de variables de entorno en un archivo `config.py``.

Documentación Técnica (README.md): Archivo explicativo en la raíz del repositorio que detalle de forma clara la arquitectura previa (legada) y la nueva arquitectura propuesta.

Fase 2: Inmutabilidad y Contenedorización Local (Docker)

Garantizar la portabilidad y el aislamiento de la aplicación mediante el uso de contenedores.

Entregables Obligatorios:

Archivos de Configuración Docker: Inclusión de un archivo Dockerfile optimizado (utilizando versiones Slim o multi-etapas) y un archivo `.dockerignore` correctamente configurado en el repositorio.

Evidencia de Funcionamiento Local: Captura de pantalla o demostración en la consola de Docker Desktop que compruebe que la imagen compila con éxito y la aplicación corre de forma aislada en el puerto configurado.

Fase 3: Pruebas Unitarias y Cobertura (Pytest)

Implementar una estrategia de pruebas que asegure la estabilidad y la calidad del código refactorizado.

Entregables Obligatorios:

Suite de Pruebas Unitarias: Código fuente de al menos 15 pruebas unitarias que validen exhaustivamente las rutas HTML y los endpoints JSON de la aplicación.

Reporte de Cobertura : Captura de la terminal o reporte generado por `pytest-cov` que demuestre que todas las pruebas fueron aprobadas y que la cobertura de código es igual o superior al 80%

Fase 4: Pruebas de Interacción Real (Selenium E2E)

Validar los flujos críticos de la aplicación desde la perspectiva del usuario final mediante pruebas de extremo a extremo.

Entregables Obligatorios:

Scripts de Automatización E2E: Desarrollo de 3 o 4 pruebas End-to-End utilizando Selenium que simulen interacciones reales (ej. inicio de sesión, creación, búsqueda y verificación de registros en interfaces).

Evidencia de Ejecución: Demostración de la ejecución exitosa local en modo visual con todas las aserciones de Selenium validadas en verde.

Fase 5: Pipeline de Integración Continua (GitHub Actions)

Automatizar el ciclo de vida del software para garantizar que cada cambio en el código sea seguro y estable.

Entregables Obligatorios:

Archivo del Pipeline: Configuración del flujo de trabajo. El script del robot debe ejecutar automáticamente las pruebas unitarias, arrancar los tests de Selenium, compilar la imagen de Docker y generar los reportes de seguridad correspondientes.

Historial de Ejecución: Enlace directo al historial de ejecuciones de *GitHub Actions* donde se evidencie el check verde de éxito del pipeline completo.

Fase 6: Entrega, Despliegue de Valor y Monitoreo

Objetivo: Liberar el producto modernizado en un entorno de producción accesible en la nube.

Entregables Obligatorios:

Configuración y Monitoreo: Implementación de un endpoint de diagnóstico `/api/health` para monitorear la salud del servidor en tiempo real.

Enlace Público de Producción: URL activa para la evaluación funcional del producto final por parte del profesor.

Como parte de la entrega final y para simular el día a día de un ingeniero de software profesional, cada alumno deberá realizar una presentación corta en vivo donde demuestre los siguientes hitos:

1. Mostrar la aplicación modernizada operando plenamente desde el enlace público.
2. Realizar un cambio menor en el código en tiempo real, ejecutar un `git push` y proyectar ante la audiencia cómo el pipeline de GitHub Actions se activa de forma autónoma, ejecuta la suite completa de pruebas (Pytest y Selenium) y actualiza el servidor de producción automáticamente.

LAS PRESENTACIONES INICIARAN UNA SEMANA ANTES DE LOS EXAMENES FINALES